

AI蛋白质设计-使用 NumPy 的神经网络



crazypig

江南大学 发酵工程博士在读

+ 关注他

这是基于[DL4Proteins](#)教程的系列学习和分享的第一篇博客，DL4Proteins教程的完整内容参照[本系列的介绍](#)。作为这个系列教程的开篇，今天我们将从最基础的神经网络开始，使用NumPy手动实现一个简单的神经网络模型。

介绍并编码我们的第一个神经元

在人工智能（AI）和机器学习的领域，神经元是构成神经网络的基本组件。AI神经元的概念受生物神经元启发，并通过模拟生物神经元的信息处理方式执行复杂的任务，如图像识别、自然语言处理和语音识别等。

神经元在神经网络中承担着基本的计算任务。一个神经元接收到来自前一层神经元的输入信号，这些信号在通过神经元时会经历一系列的数学计算。以下是人工神经元处理信号的过程：

- 输入信号**：每个神经元从上一层的神经元接收多个输入信号，这些信号代表着前一层神经元的输出值。
- 加权求和**：每个输入信号都有一个对应的**权重**，这个权重反映了该输入信号对神经元输出的重要程度。神经元会将所有输入信号与其相应的权重相乘，然后加总，得到加权和。

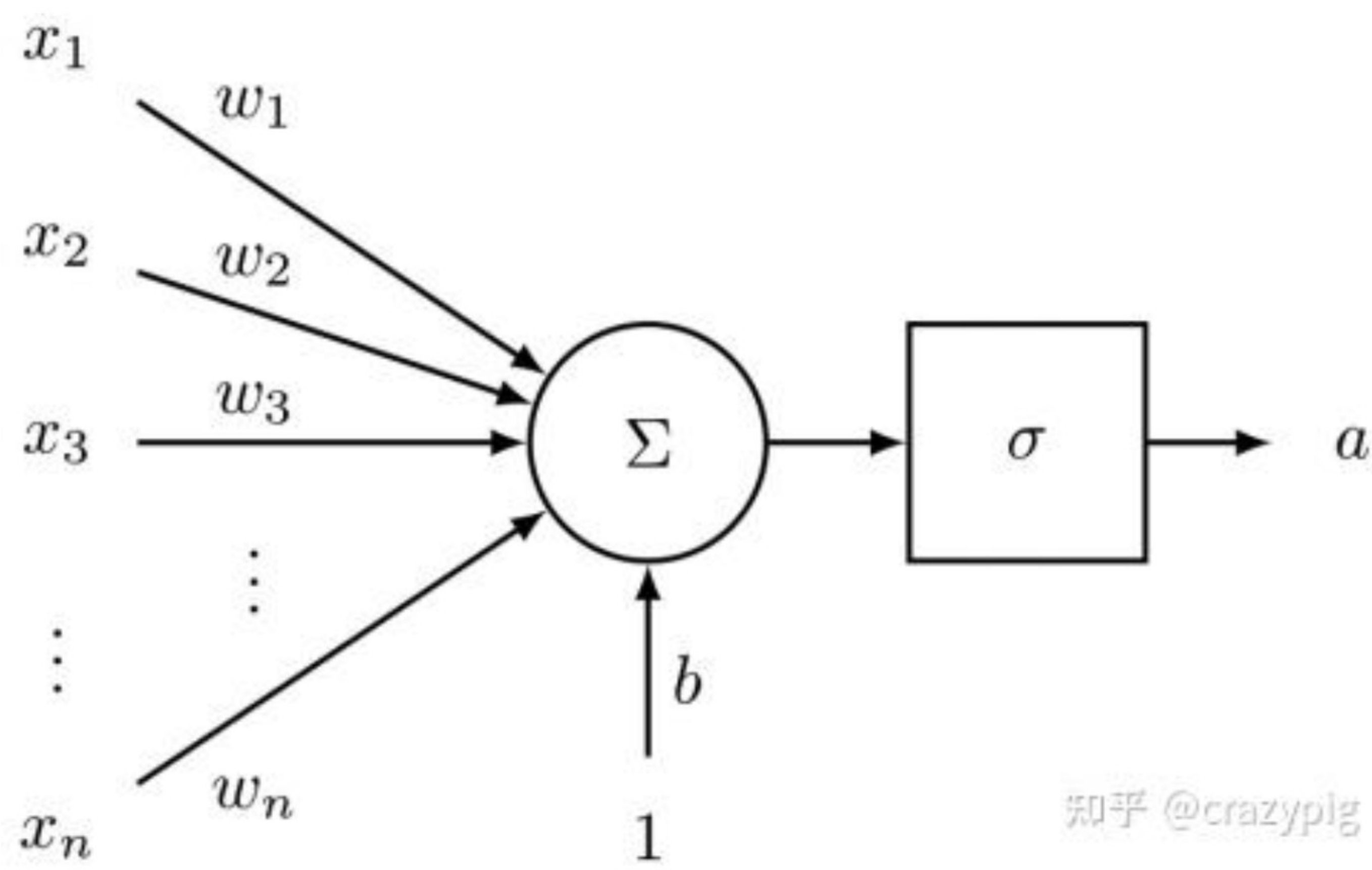
$$Z = \sum_{i=1}^n w_i \times a_i + b$$

- 激活函数**：加权和 Z 会通过一个激活函数进行处理。激活函数的作用是对加权和进行非线性变换，使得神经网络能够学习到复杂的模式。常见的激活函数有：

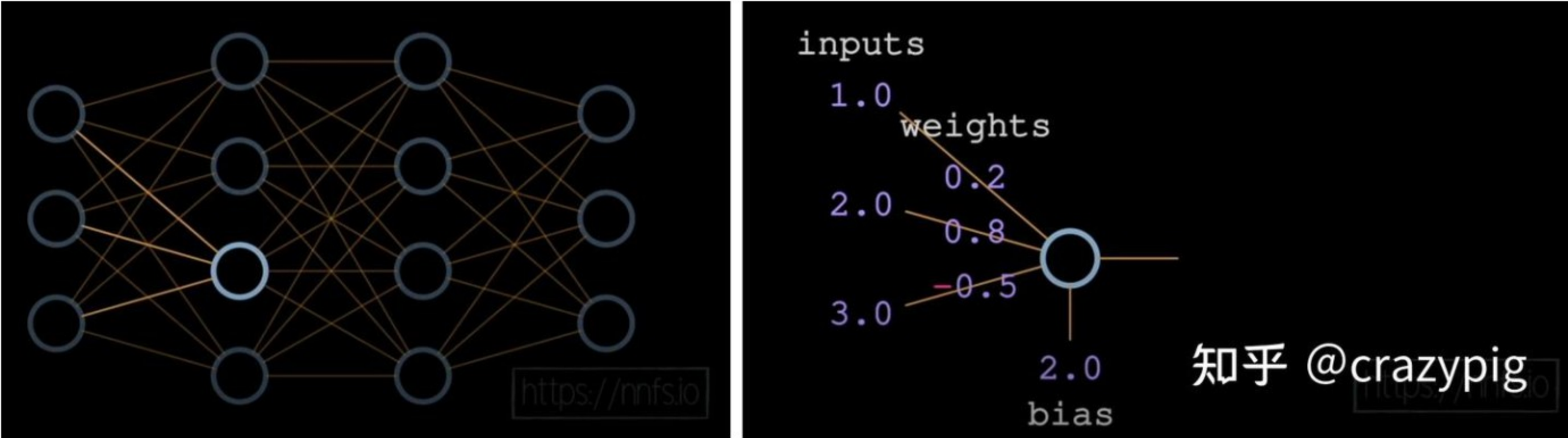
- Sigmoid函数**：将输出压缩到0和1之间，常用于二分类问题。
- ReLU (Rectified Linear Unit) 函数**：将负值输出为0，正值保持不变，常用于深度学习中的隐藏层。
- Tanh函数**：将输出压缩到-1和1之间。

$$a = \sigma(Z)$$

其中， $\sigma(Z)$ 是激活函数， a 是激活后的输出。



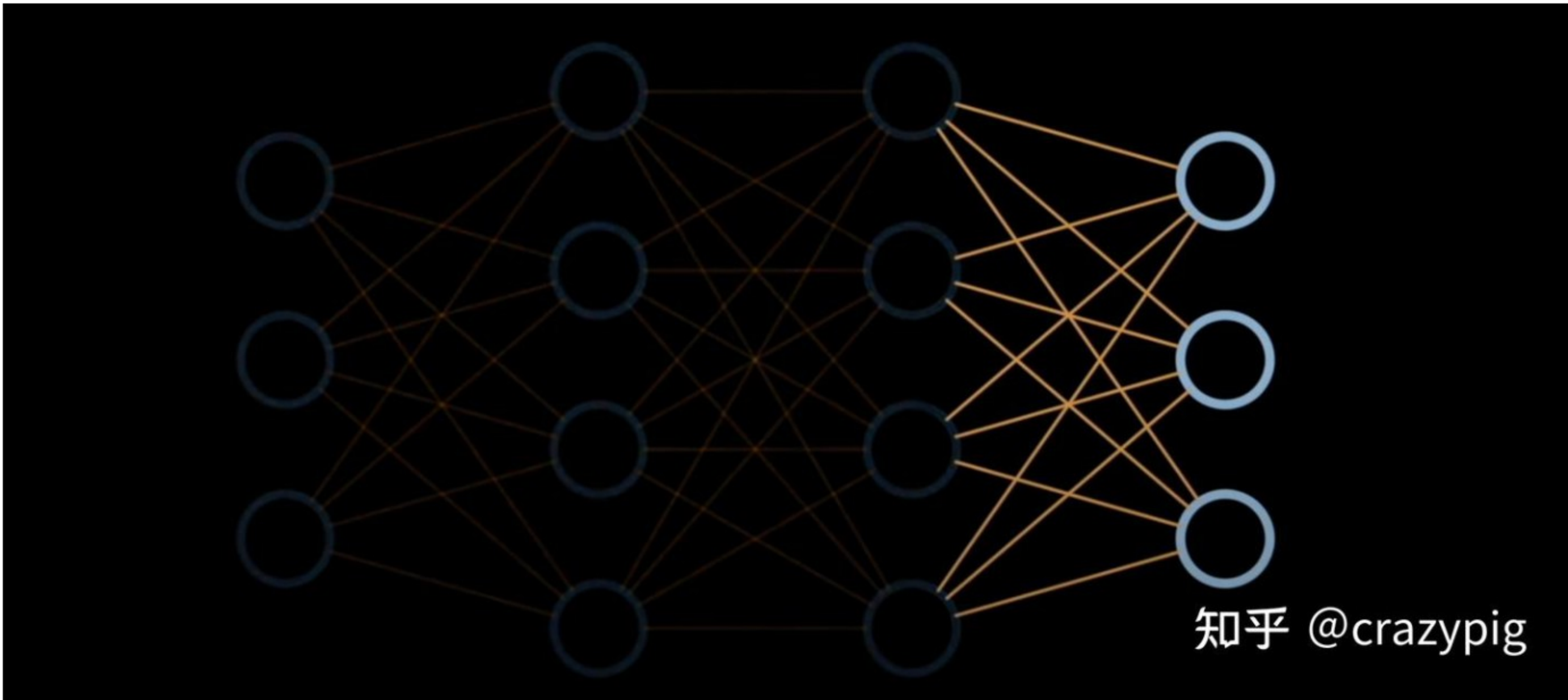
假设我们有来自前一层的 3 个神经元输入到我们正在构建的神经元。根据上述的计算规则，最终神经元的输出为2.3。



```
inputs = np.array([1, 2, 3])
weights = np.array([0.2, 0.8, -0.5])
bias = 2
output = inputs[0] * weights[0] + inputs[1] * weights[1] + inputs[2] * weights
print(output)  # 2.3
```

编码一层神经元

那么如何建模 3 个神经元，每个神经元有 4个输入呢？



由于有三个神经元，因此将有 3 个独特的权重集。每个权重集将有 4 个值（因为有 4 个输入）。我们还需要 3 个偏差。最终得到3个神经元的输出4.8, 1.21, 2.385。我们应该清楚地认识到，对于神经元的 `weights` 和 `biases` ，它们就像神经网络中的旋钮，我们可以调整它们来改变所有其他神经元之间的通信。

```
inputs = np.array([1, 2, 3, 2.5])

weights1 = np.array([0.2, 0.8, -0.5, 1.0])
bias1 = 2

weights2 = np.array([0.5, -0.91, 0.26, -0.5])
bias2 = 3

weights3 = np.array([-0.26, -0.27, 0.17, 0.87])
bias3 = 0.5

output = [inputs[0] * weights1[0] + inputs[1] * weights1[1] + inputs[2] * weights1[2] + inputs[3] * weights1[3] + bias1,
          inputs[0] * weights2[0] + inputs[1] * weights2[1] + inputs[2] * weights2[2] + inputs[3] * weights2[3] + bias2,
          inputs[0] * weights3[0] + inputs[1] * weights3[1] + inputs[2] * weights3[2] + inputs[3] * weights3[3] + bias3]

print(output) # [4.8, 1.21, 2.385]
```

我们可以使用 `numpy` 提供的点积运算来简化上面的代码

对于向量 a 和 b 的点积运算，我们记：

$$a \cdot b = \sum_{i=1}^n a_i \times b_i$$


```
inputs = np.array([1, 2, 3, 2.5])

weights = np.array([[0.2, 0.8, -0.5, 1.0], [0.5, -0.91, 0.26, -0.5], [-0.26, -0.5, 0.8, 0.5]])
bias = [2, 3, 0.5]

output = np.dot(weights, inputs) + bias
print(output) # [4.8, 1.21, 2.385]
```

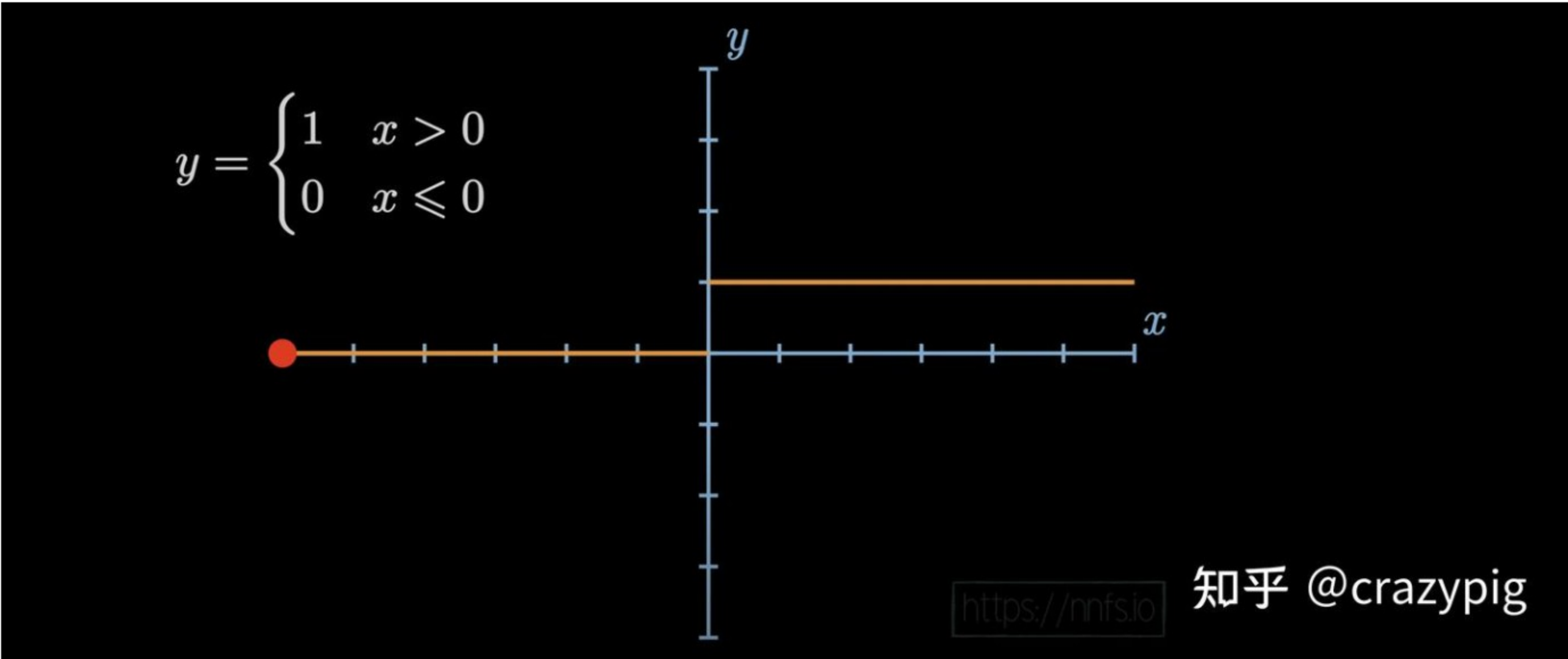
隐藏层激活函数

激活函数在 hidden layers （输入层或输出层以外的层）中用于多种原因。它们有助于规范化每个神经元的输出，从而防止值在通过网络时变得过大或过小。

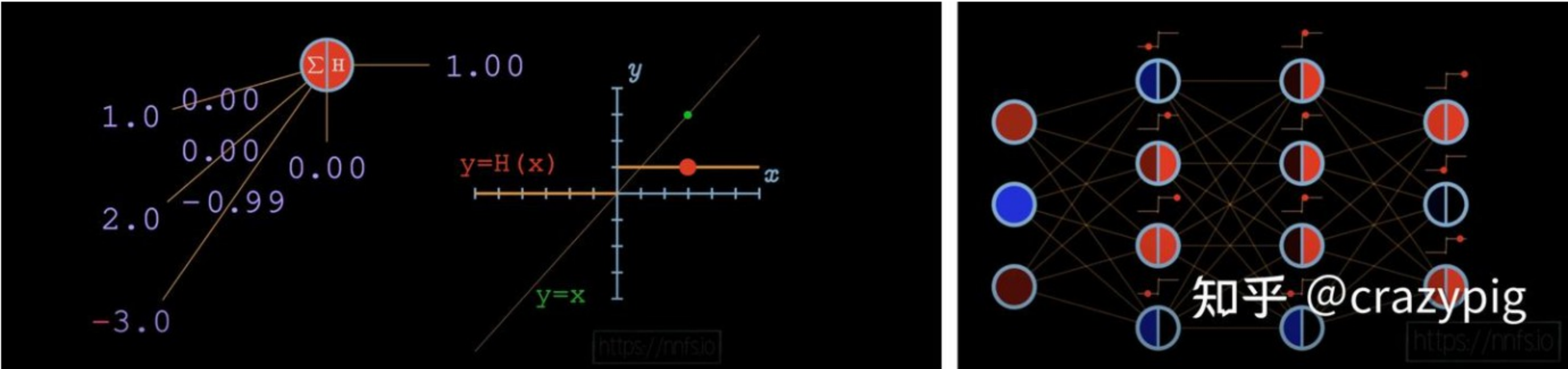
这里我们将讨论 3 个激活函数

- 1. 阶跃函数
- 2. Sigmoid函数
- 3. ReLU （Rectified Linear Unit） 函数

在阶跃函数中，如果输入大于 0，则输出为 1，否则输出为 0。



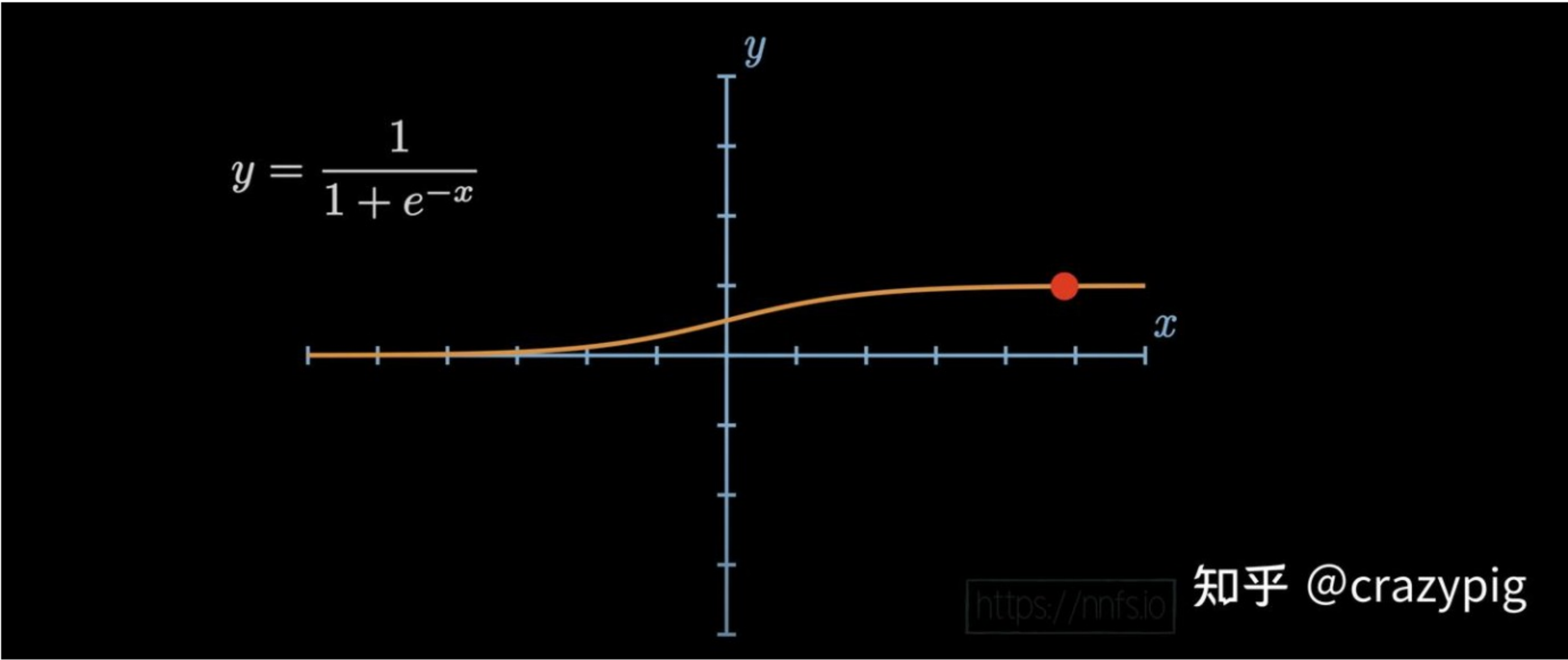
因此，在阶跃函数的情况下，神经元的输出实际上是 0 或 1。



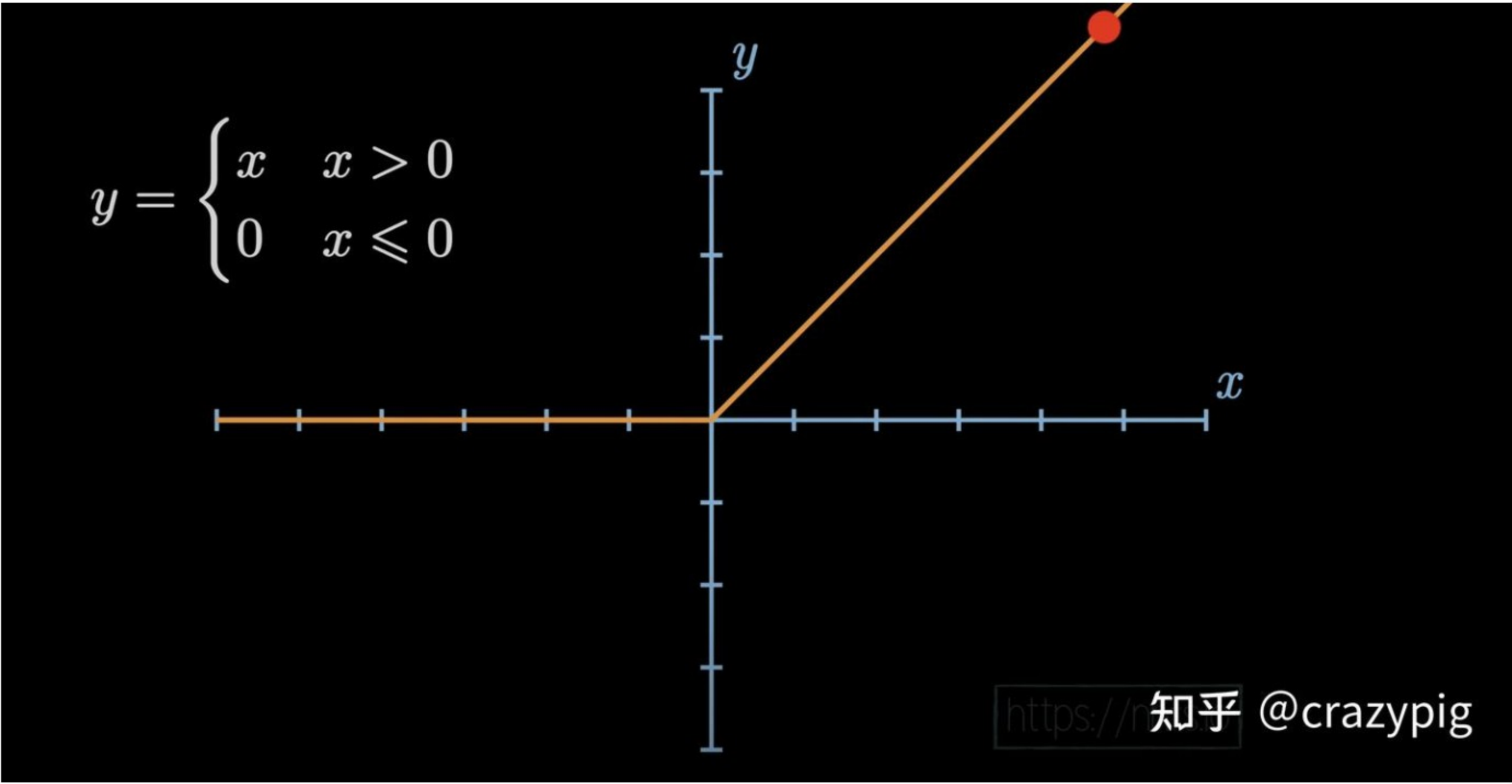
虽然这在理论上是可行的，但很快就会发现，使用更精细的参数将使神经网络训练得更好。例如，在阶跃函数中，如果输出为 0，我们如何知道输入值距离输出 1 还有多远？了解这一点将有助于模

型更好地自我修正，并在计算损失和优化权重和偏差时发挥作用。

一个这样的函数就是**Sigmoid函数**。我们可以很容易地看到输出与 0（或 1）的接近程度（或远离程度）。然而，即使是Sigmoid函数也不是完美的，它**梯度消失**的缺点在我们讨论梯度之前不做讨论。



最后是**ReLU（Rectified Linear Unit）函数**，如果 x 大于 0，则输出为 x，否则输出为 0。我们使用 ReLU 的主要原因之一是它速度快：如果 x 小于 0，则输出 0，如果 x 大于 0，则输出 x。



你可能会问为什么我们不使用线性激活函数（比如 $y=x$ ），但用线性函数对非线性数据进行建模是不可能的，所以我们确实需要非线性激活函数。

输出层激活函数

为什么要使用另一个激活函数？我们想要一个函数来帮助量化相对的“正确性”。在下面的两个输出层中，哪一个相对更正确？假设数组中的第一个节点是正确的预测，那么 `layer_outputs1` 可能更正确，因为它相对大于其他数组值。


```
layer_outputs1 = np.array([4.8, 1.21, 2.385])
layer_outputs2 = np.array([4.8, 4.79, 4.25])
```

softmax函数通常用于机器学习中的多类分类任务。它以任意实值分数向量（通常称为 logits）作为输入，并将其归一化为多个类别的概率分布。

k个元素的向量z的softmax函数定义为：

$$softmax(z)_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

其中：

- $softmax(z)_i$ 是第i个元素的结果概率分布
- e 是自然对数的底数
- z_i 是向量z的第i个元素
- 分母是所有类别的指数分数的总和

softmax函数确保输出概率总和为 1，这使其适用于模型需要输出类概率的分类任务。下面计算 layer_outputs1 和 layer_outputs2 的 softmax。

```
def calculate_softmax(layer_output):
    softmax = np.exp(layer_output) / np.sum(np.exp(layer_output))
    return softmax

layer_outputs1 = np.array([4.8, 1.21, 2.385])
layer_outputs2 = np.array([4.8, 4.79, 4.25])

s1 = calculate_softmax(layer_outputs1)
sum1 = np.sum(s1)
s2 = calculate_softmax(layer_outputs2)
sum2 = np.sum(s2)
print(s1, sum1, s2, sum2)
# [0.89528266 0.02470831 0.08000903] 0.9999999999999999 [0.38955985 0.38568367
```

损失的计算和执行

我们如何优化我们的模型？我们是否应该优化预测标签与真实标签的准确性？这是一个很好的策略，但是你将丢弃很多有用的信息，因为你的输出是一个概率分布而不是一个数字。

分类交叉熵是神经网络分类模型中常用的损失函数，尤其是对于多类分类任务。它衡量标签的真实概率分布与模型输出的预测概率分布之间的差异。此损失惩罚真实概率和预测概率之间的较大误差，鼓励模型为正确的类标签输出更高的概率。

$$L(y, \hat{y}) = - \sum_{i=1}^K y_i \log(\hat{y}_i)$$

其中：

- $L(y, \hat{y})$ 是分类交叉熵损失
- K 是类别数量.
- y 真实onhot编码的标签
- $\hat{y}$ 是模型输出的预测概率分布
- y_i 是属于类别i的真实概率
- $\hat{y}_i$ 是属于类别i的预测概率

```
softmax_output = np.array([0.7, 0.1, 0.2])
target_output = np.array([1, 0, 0])

target_class = 0

loss = -(math.log(softmax_output[0]) * target_output[0] +
          math.log(softmax_output[1]) * target_output[1] +
          math.log(softmax_output[2]) * target_output[2])

print(loss)  # 0.35667494393873245
```

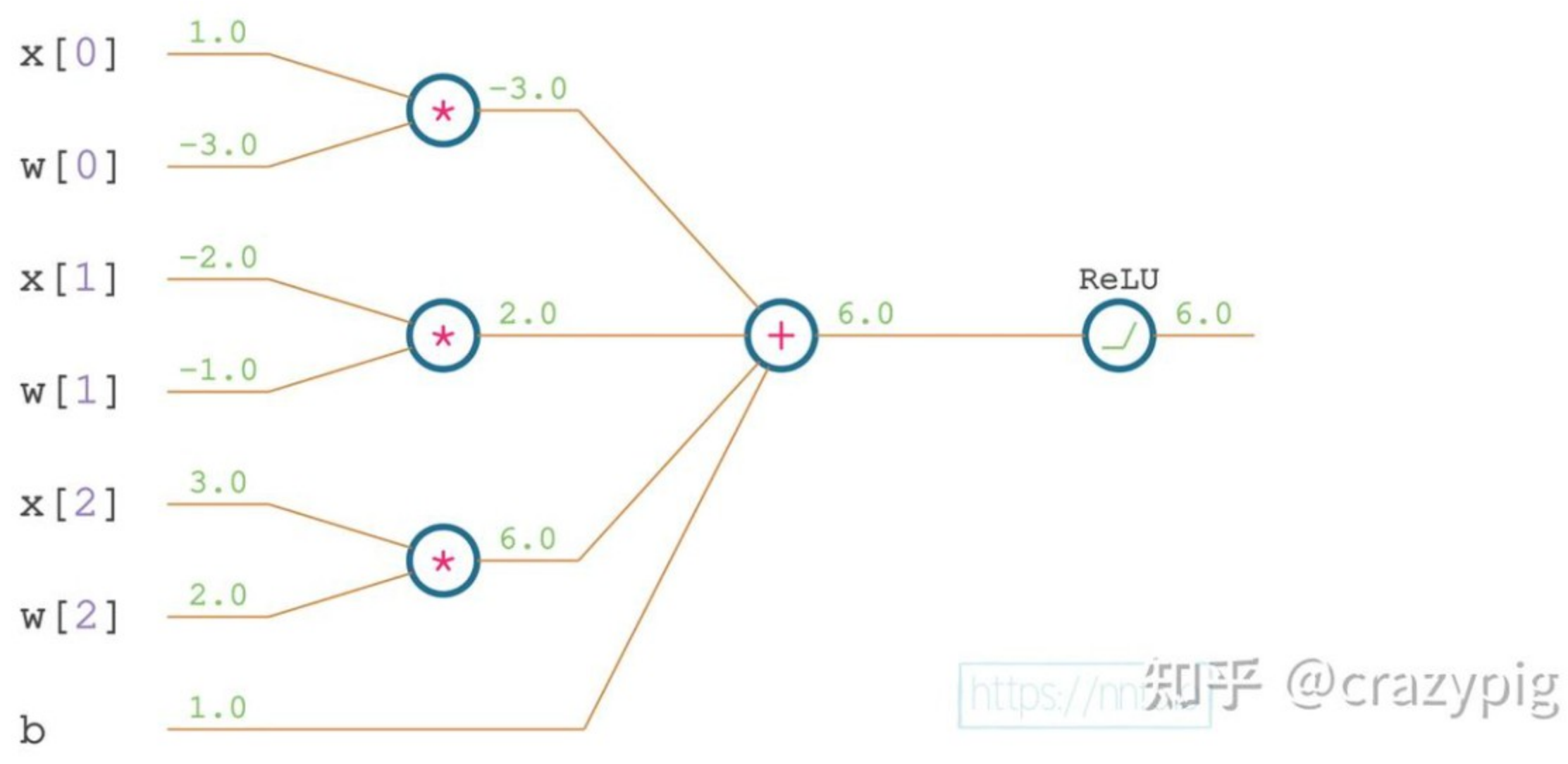
反向传播

随机搜索权重和偏差是低效的。为了知道如何调整权重和偏差，我们首先需要了解它们对损失的影响。为了了解所有输入、权重和偏差对神经元输出的影响，在损失函数的最后，我们需要计算神经元和整个模型前向传递过程中执行的每个操作的导数。

我们需要了解每个权重和偏差如何影响损失，以改善损失。要对函数链执行此操作，请使用链式法则，该法则表示函数链的导数是此链中所有函数导数的乘积。

$$\frac{\partial}{\partial x} f(g(y, h(x, z))) = \frac{\partial f(g(y, h(x, z)))}{\partial g(y, h(x, z))} \cdot \frac{\partial g(y, h(x, z))}{\partial h(x, z)} \cdot \frac{\partial h(x, z)}{\partial x}$$

让我们反向传播单个神经元的 ReLU 函数，并假设我们打算最小化该单个神经元的输出。我们将利用带有导数和偏导数的链式法则来计算每个变量对 ReLU 激活输出的影响。



上图中神经元的传播过程可以写成如下的数学形式

$$y = max(\sum_{i=0}^2 w_i \times x_i + b, 0)$$

根据链式法则，我们有

$$z = \sum_{i=0}^2 w_i \times x_i + b$$
$$\frac{dy}{dw_i} = \frac{dy}{dz} \frac{dz}{dw_i} = \frac{d \max(z, 0)}{dz} x_i$$
$$\frac{dy}{db} = \frac{dy}{dz} \frac{dz}{db} = \frac{d \max(z, 0)}{dz}$$

通过取 ReLU() 导数、求和运算的导数、并将两者相乘来计算复合函数的导数，这就是使用链式法则进行反向传播。

优化器

我们希望使用计算出的梯度来调整权重和偏差，最终降低损失的度量。我们可以通过这种方式成功降低神经元的激活函数 (ReLU) 输出。我们通常从每个权重和偏差中减去梯度的一小部分来实现这种调整。当我们有数百万、数十亿甚至更多的维度时，**随机梯度下降 (SGD)**是搜索全局最小值的最佳方法。

学习率和动量

尽管我们希望模型尽可能准确，但优化器的目标是最小化损失，而不是直接提高准确率。损失是所有样本损失的平均值，因此虽然某些损失可能会大幅减少，但其他损失可能只会略有增加，这可能会导致错误的预测。

在训练初期，较大的更新有助于探索解决方案空间的不同区域，但随着训练的进行，我们需要更小、更精细的更新。为了实现这一点，我们实现了**学习率衰减**。通过将初始学习率除以步数，我们

确保学习率一开始迅速下降，然后逐渐减慢。这使得模型在接近最优解时能够进行微调。

在上一节中，我们介绍了**SGD**作为优化模型的一种方法。虽然它可能很有效，但它是一种遵循梯度的基本方法，无需额外的逻辑，可能有助于模型找到损失函数的全局最小值。

动量通过创建多次更新的梯度滚动平均值来解决此问题，并将该平均值与当前梯度相结合。这有助于模型超越局部最小值并进一步减少损失。

从头开始完成神经网络

让我们将学到的概念应用到一个简单的神经网络中。

定义模型

以下是我们将用来创建模型的一系列类。

```
# Dense layer
class Layer_Dense:
    """
    Dense layer of a neural network
    Facilitates:
    - Forward propogation of data throught layer
    - Backward propogation of gradients during training
    """

    # Layer initialization
    def __init__(self, n_inputs, n_neurons):
        # Initialize weights and biases
        self.weights = 0.01 * np.random.randn(n_inputs, n_neurons)
        self.biases = np.zeros((1, n_neurons))

    # Forward pass
    def forward(self, inputs):
        # Remember input values
        self.inputs = inputs
        # Calculate output values from inputs, weights and biases
        self.output = np.dot(inputs, self.weights) + self.biases

    # Backward pass
    def backward(self, dvalues):
        # Gradients on parameters
        self.dweights = np.dot(self.inputs.T, dvalues)
        self.dbiases = np.sum(dvalues, axis=0, keepdims=True)
        # Gradient on values
        self.dinputs = np.dot(dvalues, self.weights.T)
```



```
# ReLU activation
class Activation_ReLU:
    """
    Rectified linear unit activation function
    Applied to input of neural network layer
    Introduces non-linearity into the network
    """

    # Forward pass
    def forward(self, inputs):
        # Remember input values
        self.inputs = inputs
        # Calculate output values from inputs
        self.output = np.maximum(0, inputs)

    # Backward pass
    def backward(self, dvalues):
        # Since we need to modify original variable,
        # let's make a copy of values first
        self.dinputs = dvalues.copy()
        # Zero gradient where input values were negative
        self.dinputs[self.inputs <= 0] = 0

# Softmax classifier - combined Softmax activation
# and cross-entropy loss for faster backward step
class Activation_Softmax_Loss_CategoricalCrossentropy():
    """
    Combination of softmax activation function and categorical cross entropy loss
    Commonly used in classification tasks
    We minimize loss by adjusting model parameters to improve performance
    """

    # create activation and loss function objectives
    def __init__(self):
        self.activation = Activation_Softmax()
        self.loss = Loss_CategoricalCrossentropy()

    # forward pass
    def forward(self, inputs, y_true):
        # output layer's activation function
        self.activation.forward(inputs)
        # set the output
        self.output = self.activation.output
        # calculate and return loss value
        return self.loss.calculate(self.output, y_true)

    # backward pass
    def backward(self, dvalues, y_true):
```



```
# number of samples
samples = len(dvalues)

# if labels one-hot encoded, turn into discrete values
if len(y_true.shape) == 2:
    y_true = np.argmax(y_true, axis=1)

# copy so we can safely modify
self.dinputs = dvalues.copy()
# Calculate gradient
self.dinputs[range(samples), y_true] -= 1
# Normalize gradient
self.dinputs = self.dinputs / samples

# Adam optimizer
class Optimizer_Adam:

    """
    Adam optimization algorithm to optimize parameters of neural network
    Initialize with learning rate, decay, epsilon, momentum
    Pre-update params: Adjust learning rate based on decay
    Update params: Update params using momentum and cache corrections
    Post-update params: Track number of optimization steps performed
    """

    # Initialize optimizer - set settings
    def __init__(self, learning_rate=0.001, decay=0., epsilon=1e-7, beta_1=0.9, beta_2=0.999):
        self.learning_rate = learning_rate
        self.current_learning_rate = learning_rate
        self.decay = decay
        self.iterations = 0
        self.epsilon = epsilon
        self.beta_1 = beta_1
        self.beta_2 = beta_2

    # Call once before any parameter updates
    def pre_update_params(self):
        if self.decay:
            self.current_learning_rate = self.learning_rate * (1. / (1. + self.decay))

    # Update parameters
    def update_params(self, layer):

        # If layer does not contain cache arrays, create them filled with zeros
        if not hasattr(layer, 'weight_cache'):
            layer.weight_momentums = np.zeros_like(layer.weights)
            layer.weight_cache = np.zeros_like(layer.weights)
            layer.bias_momentums = np.zeros_like(layer.biases)
            layer.bias_cache = np.zeros_like(layer.biases)
```



```

# Update momentum with current gradients
layer.weight_momentums = self.beta_1 * layer.weight_momentums + (1 - self.beta_1) * layer.weight_gradients
layer.bias_momentums = self.beta_1 * layer.bias_momentums + (1 - self.beta_1) * layer.bias_gradients

# Get corrected momentum
# self.iteration is 0 at first pass
# and we need to start with 1 here
weight_momentums_corrected = layer.weight_momentums / (1 - self.beta_1 ** (self.iteration + 1))
bias_momentums_corrected = layer.bias_momentums / (1 - self.beta_1 ** (self.iteration + 1))

# update cache with squared current gradients
layer.weight_cache = self.beta_2 * layer.weight_cache + (1 - self.beta_2) * layer.weight_momentums_corrected ** 2
layer.bias_cache = self.beta_2 * layer.bias_cache + (1 - self.beta_2) * layer.bias_momentums_corrected ** 2

# get corrected cache
weight_cache_corrected = layer.weight_cache / (1 - self.beta_2 ** (self.iteration + 1))
bias_cache_corrected = layer.bias_cache / (1 - self.beta_2 ** (self.iteration + 1))

# Vanilla SGD parameter update + normalization with square root cache
layer.weights += -self.current_learning_rate * weight_momentums_corrected / (weight_cache_corrected ** 0.5)
layer.biases += -self.current_learning_rate * bias_momentums_corrected / (bias_cache_corrected ** 0.5)

# call once after any parameter updates
def post_update_params(self):
    self.iterations += 1

# Softmax activation
class Activation_Softmax:

    """
    Softmax activation function for multi-class classification
    Compute probabilities for each class
    """

    # Forward pass
    def forward(self, inputs):
        # Remember input values
        self.inputs = inputs
        # Get unnormalized probabilities
        exp_values = np.exp(inputs - np.max(inputs, axis=1, keepdims=True))

        # Normalize them for each sample
        probabilities = exp_values / np.sum(exp_values, axis=1, keepdims=True)

        self.output = probabilities

```



```
# Backward pass
```

```
def backward(self, dvalues):
```

```
    # Create uninitialized array
```

```
    self.dinputs = np.empty_like(dvalues)
```

```
    # Enumerate outputs and gradients
```

```
    for index, (single_output, single_dvalues) in enumerate(zip(self.output, dvalues)):
```

```
        # Flatten output array
```

```
        single_output = single_output.reshape(-1, 1)
```

```
        # Calculate Jacobian matrix of the output
```

```
        jacobian_matrix = np.diagflat(single_output) - np.dot(single_output, single_dvalues)
```

```
        # Calculate sample-wise gradient and add it to the array of sample gradients
```

```
        self.dinputs[index] = np.dot(jacobian_matrix, single_dvalues)
```

```
# Common loss class
```

```
class Loss:
```

```
    # calculates data and regularization losses, given model output and ground truth
```

```
    def calculate(self, output, y):
```

```
        # calculate sample losses
```

```
        sample_losses = self.forward(output, y)
```

```
        # calculate mean losses
```

```
        data_loss = np.mean(sample_losses)
```

```
        # return loss
```

```
        return data_loss
```

```
# cross entropy loss
```

```
class Loss_CategoricalCrossentropy(Loss):
```

```
    """
```

```
    Computes categorical cross entropy
```

```
    Quantifies discrepancy between predicted and true class probabilities
```

```
    """
```

```
    # forward pass
```

```
    def forward(self, y_pred, y_true):
```

```
        # number samples in batch
```

```
        samples = len(y_pred)
```

```
        # clip data to prevent division by 0
```

```
        # clip both sides to not drag mean towards any value
```

```
        y_pred_clipped = np.clip(y_pred, 1e-7, 1 - 1e-7)
```



```
# probabilities for target values (only if categorical labels)
if len(y_true.shape) == 1:
    correct_confidences = y_pred_clipped[ range(samples), y_true ]

# mask values (only for one-hot encoded labels)
elif len(y_true.shape) == 2:
    correct_confidences = np.sum( y_pred_clipped * y_true, axis=1 )

# losses
negative_log_likelihoods = -np.log(correct_confidences)
return negative_log_likelihoods

# backward pass
def backward(self, dvalues, y_true):

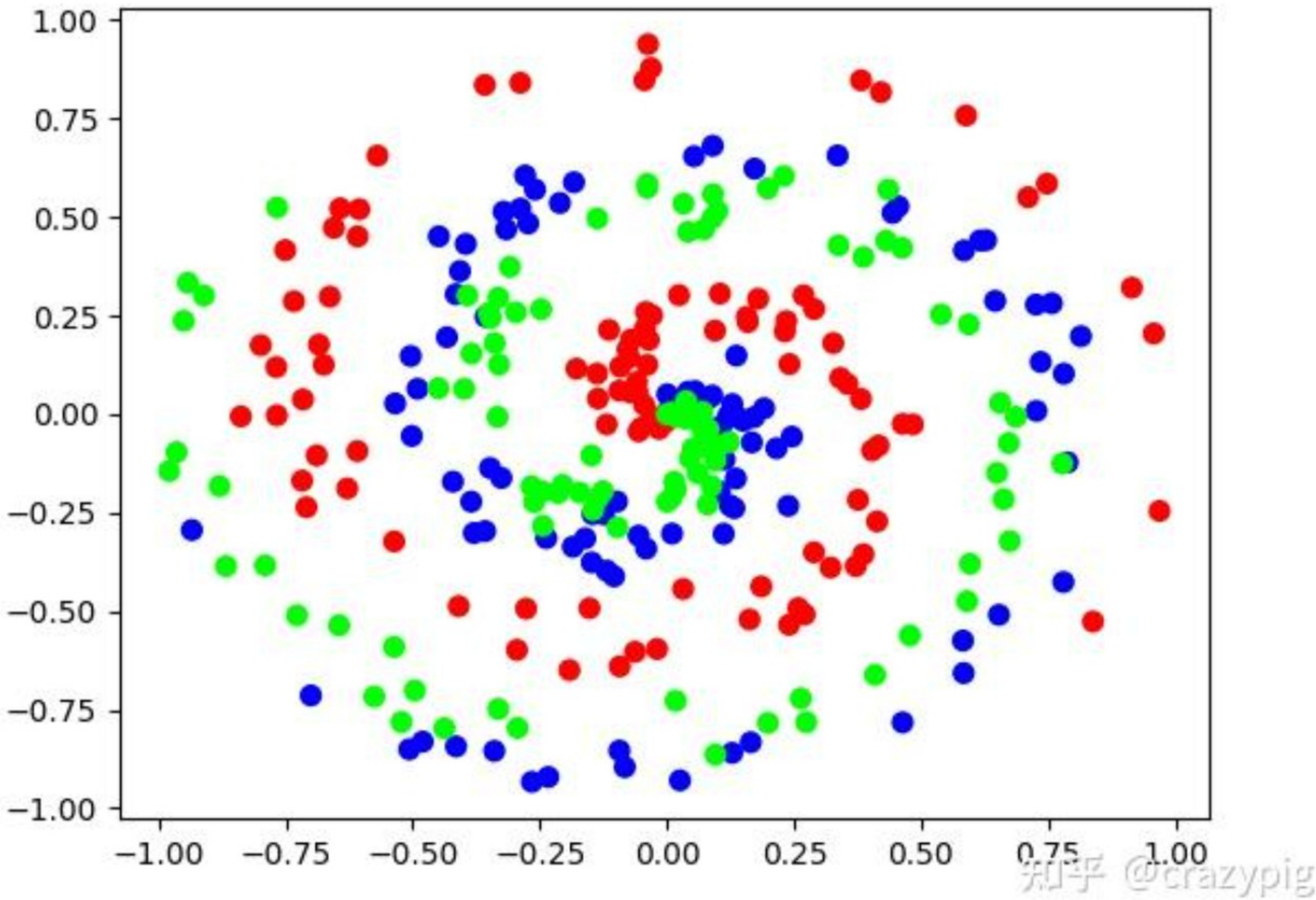
    # number of samples
    samples = len(dvalues)
    # Number of labels in every sample
    # We'll use the first sample to count them
    labels = len(dvalues[0])

    if len(y_true.shape) == 1:
        y_true = np.eye(labels)[y_true]

    # calculate gradient
    self.dinputs = -y_true / dvalues
    # Normalize gradient
    self.dinputs = self.dinputs / samples
```

创建数据集

我们给定一个两个特征（x 和 y 坐标）的输入来训练一个神经网络来预测一个点的颜色（y ∈ {green, red, or blue}）。

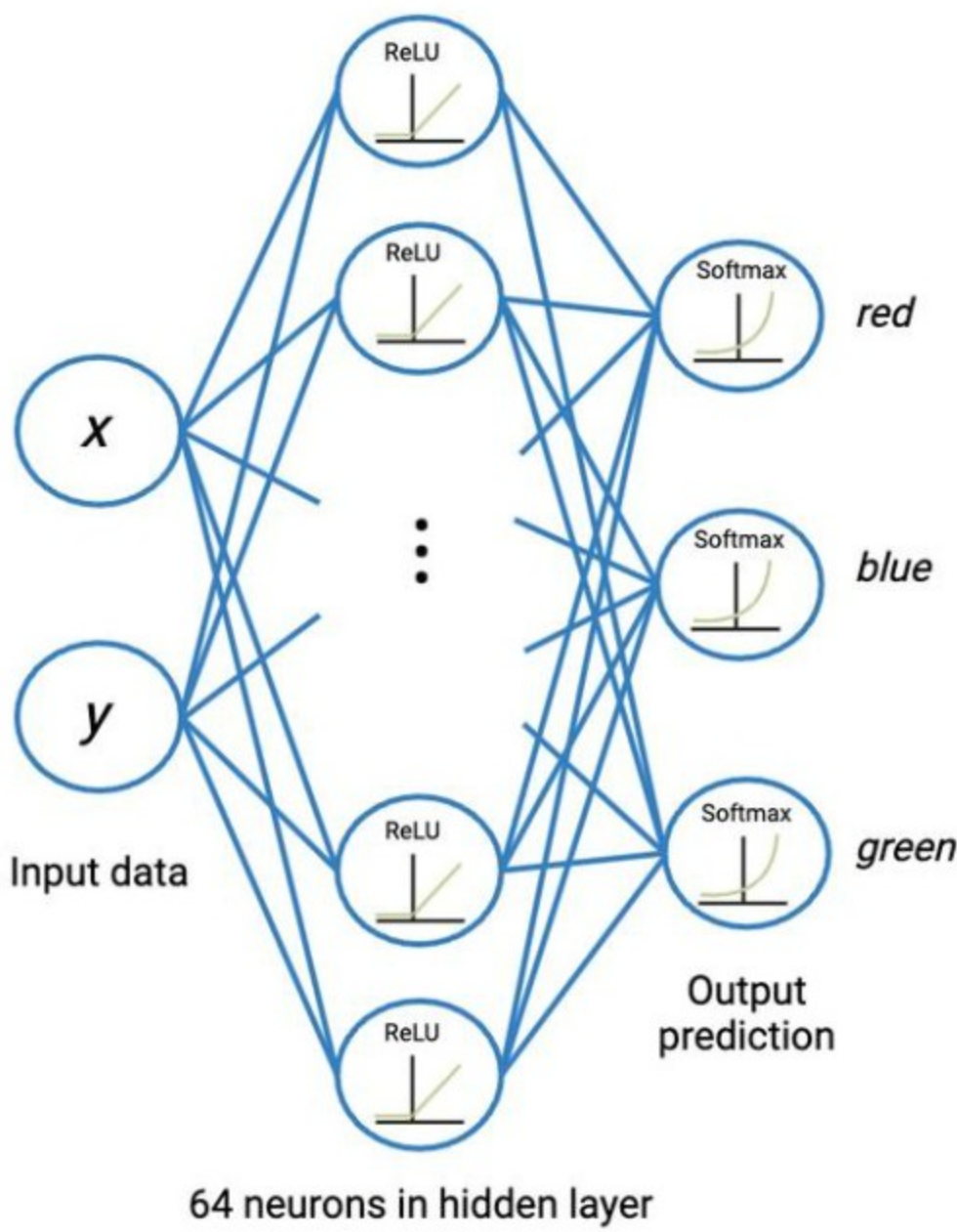


```
# Create dataset
X, y = spiral_data(samples=100, classes=3)

plt.scatter(X[:, 0], X[:, 1], c=y, s=40, cmap='brg')
```

实例化模型

我们将使用一个由两个带有 ReLU 激活的全连接层组成的前馈神经网络，其中第一层采用 2 个输入特征并输出 64 个神经元，第二层采用 64 个神经元并输出 3 个神经元。



Loss
Categorical cross entropy

$$L(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^K y_{ij} \log(\hat{y}_{ij})$$

Optimizer
Adam optimization

知乎 @crazypig


```
# Create Dense layer with 2 input features and 64 output values
dense1 = Layer_Dense(2, 64)

# Create ReLU activation (to be used with Dense layer):
activation1 = Activation_ReLU()

# Create second Dense layer with 64 input features (as we take output
# of previous layer here) and 3 output values (output values)
dense2 = Layer_Dense(64, 3)

# Create Softmax classifier's combined loss and activation
loss_activation = Activation_Softmax_Loss_CategoricalCrossentropy()

# Create optimizer
optimizer = Optimizer_Adam(learning_rate=0.05, decay=5e-7)
```

训练

```
# Train in loop
for epoch in range(10001):
    # Perform a forward pass of our training data through this layer
    dense1.forward(X)

    # Perform a forward pass through activation function
    # takes the output of first dense layer here
    activation1.forward(dense1.output)

    # Perform a forward pass through second Dense layer
    # takes outputs of activation function of first layer as inputs
    dense2.forward(activation1.output)

    # Perform a forward pass through the activation/loss function
    # takes the output of second dense layer here and returns loss
    loss = loss_activation.forward(dense2.output, y)

    # Calculate accuracy from output of activation2 and targets
    # calculate values along first axis
    predictions = np.argmax(loss_activation.output, axis=1)

    if len(y.shape) == 2:
        y = np.argmax(y, axis=1)

    accuracy = np.mean(predictions==y)

    if not epoch % 100:
        print(f'epoch: {epoch}, ' + f'acc: {accuracy:.3f}, ' + f'loss: {loss:.3f},
```



```
# backward pass
loss_activation.backward(loss_activation.output, y)
dense2.backward(loss_activation.dinputs)
activation1.backward(dense2.dinputs)
dense1.backward(activation1.dinputs)

# update weights and biases
optimizer.pre_update_params()
optimizer.update_params(dense1)
optimizer.update_params(dense2)
optimizer.post_update_params()
```

可视化预测

```
import matplotlib.pyplot as plt

# Assuming you have access to the weights and biases of your trained model
# For the weights and biases of dense1 and dense2 layers
W1, b1 = dense1.weights, dense1.biases
W2, b2 = dense2.weights, dense2.biases

# Create a meshgrid of points covering the feature space
h = 0.02
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

# Flatten the meshgrid points and apply the first dense layer and ReLU activation
points = np.c_[xx.ravel(), yy.ravel()]
z1 = np.dot(points, W1) + b1
a1 = np.maximum(0, z1)

# Apply the second dense layer
z2 = np.dot(a1, W2) + b2

# Apply softmax activation to get probabilities
exp_scores = np.exp(z2 - np.max(z2, axis=1, keepdims=True))
probs = exp_scores / np.sum(exp_scores, axis=1, keepdims=True)

# Predictions
predictions = np.argmax(probs, axis=1)
Z = predictions.reshape(xx.shape)

# Plot decision boundary
plt.contourf(xx, yy, Z, cmap='brg', alpha=0.8)
```



```
# Plot data points
plt.scatter(X[:, 0], X[:, 1], c=y, s=40, cmap='brg')
plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.show()
```

